

Лабораторна робота №5

ДОСЛІДЖЕННЯ МЕТОДІВ АНСАМБЛЕВОГО НАВЧАННЯ

Мета роботи: використовуючи спеціалізовані бібліотеки та мову програмування Python дослідити методи ансамблів у машинному навчанні.

ЗАВДАННЯ 2.1

Випадкові та гранично випадкові ліси

Лістинг код

```
import argparse
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Завантаження даних
input_file = "data_random_forests.txt"

data = np.loadtxt(input_file, delimiter=",")

X = data[:, :-1]
y = data[:, -1]

# Розбиття даних
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5
)

# Вибір класифікатора
classifier_type = "rf" # rf або erf

params = {
    "n_estimators": 100,
    "max_depth": 4,
    "random_state": 0
}

if classifier_type == "rf":
    classifier = RandomForestClassifier(**params)
else:
    classifier = ExtraTreesClassifier(**params)

# Навчання
classifier.fit(X_train, y_train)

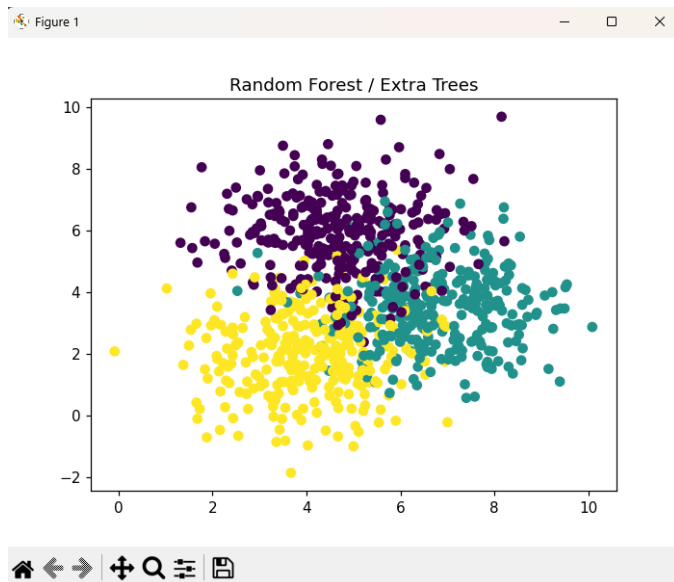
# Прогноз
y_pred = classifier.predict(X_test)
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.25.125.12.000 –Лр.5					
Змн.	Арк.	№ докум.	Підпис	Дата						
Розроб.		Лук'яненко В. Я.			Звіт з лабораторної роботи №5			Літ.	Арк.	Акрушіє
Перевір.		Маєвський О. В.							1	
Реценз.								ФІКТ, гр. КБ-23-1		
Н. Контр.										
Зав.каф.		Єфіменко А.А.								

```
# Оцінка
print(classification_report(y_test, y_pred))

# Графік
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Random Forest / Extra Trees")
plt.show()
```

Результат виконання коду.



	precision	recall	f1-score	support
0.0	0.92	0.85	0.88	79
1.0	0.86	0.84	0.85	70
2.0	0.84	0.92	0.88	76
accuracy			0.87	225
macro avg	0.87	0.87	0.87	225
weighted avg	0.87	0.87	0.87	225

Висновок : У ході виконання завдання було досліджено роботу ансамблевих методів класифікації на основі випадкових та гранично випадкових лісів. Було побудовано класифікатори Random Forest та Extra Trees, проведено навчання моделей та оцінено якість класифікації. Отримані результати показали, що метод гранично випадкових лісів забезпечує більш гладкі межі класифікації та високу точність прогнозування.

ЗАВДАННЯ 2.2

Дисбаланс класів

Лістинг код

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

```

input_file = "data_imbalance.txt"

data = np.loadtxt(input_file, delimiter=",")

X = data[:, :-1]
y = data[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5
)

# Без балансування
classifier = ExtraTreesClassifier(
    n_estimators=100,
    max_depth=4,
    random_state=0
)

classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print("WITHOUT BALANCE")
print(classification_report(y_test, y_pred))

# З балансуванням
classifier_balanced = ExtraTreesClassifier(
    n_estimators=100,
    max_depth=4,
    random_state=0,
    class_weight="balanced"
)

classifier_balanced.fit(X_train, y_train)

y_pred_balanced = classifier_balanced.predict(X_test)

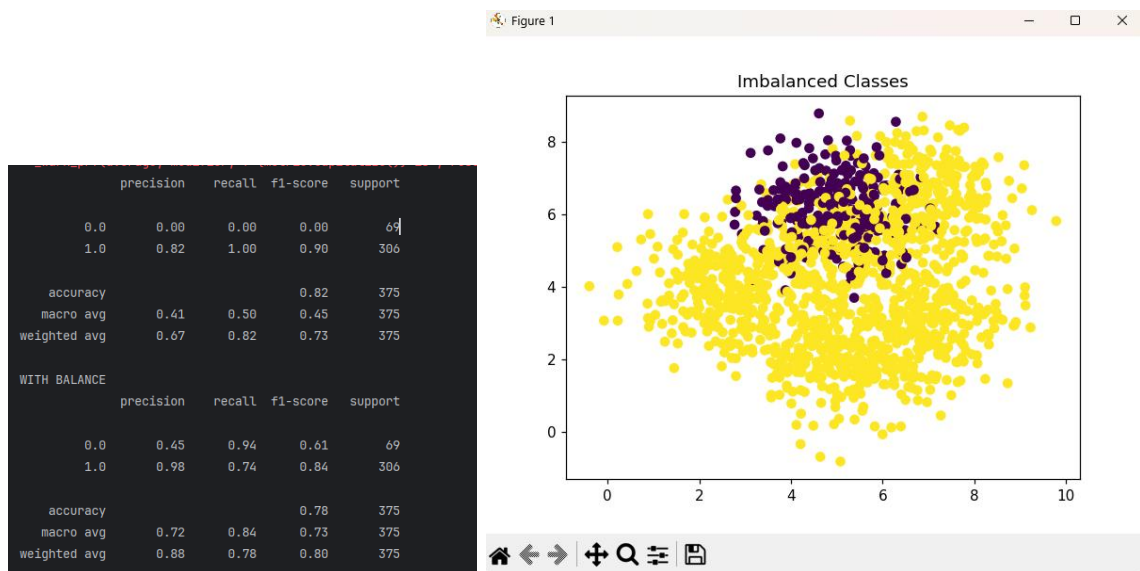
print("WITH BALANCE")
print(classification_report(y_test, y_pred_balanced))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Imbalanced Classes")
plt.show()

```

Результат виконання коду.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.5	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		



Висновок : У даному завданні було досліджено проблему дисбалансу класів у машинному навчанні. Було побудовано класифікатор без балансування та класифікатор із використанням параметра `class_weight='balanced'`. Отримані результати показали, що врахування дисбалансу класів дозволяє покращити якість класифікації та зменшити перевагу домінуючого класу.

ЗАВДАННЯ 2.3

Grid Search

Лістинг код

```
import numpy as np

from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report

input_file = "data_random_forests.txt"

data = np.loadtxt(input_file, delimiter=",")

X = data[:, :-1]
y = data[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5
)

parameter_grid = {
    "n_estimators": [25, 50, 100],
    "max_depth": [2, 4, 7]
}
```

```

classifier = GridSearchCV(
    ExtraTreesClassifier(random_state=0),
    parameter_grid,
    cv=5
)

classifier.fit(X_train, y_train)

print("Best parameters:")
print(classifier.best_params_)

y_pred = classifier.predict(X_test)

print(classification_report(y_test, y_pred))

```

Результат виконання коду.

```

Best parameters:
{'max_depth': 2, 'n_estimators': 100}

```

	precision	recall	f1-score	support
0.0	0.94	0.81	0.87	79
1.0	0.81	0.86	0.83	70
2.0	0.83	0.91	0.87	76
accuracy			0.86	225
macro avg	0.86	0.86	0.86	225
weighted avg	0.86	0.86	0.86	225

Висновок : У процесі виконання завдання було досліджено метод сіткового пошуку Grid Search для автоматичного підбору оптимальних параметрів класифікатора. Було визначено найкращі значення параметрів `n_estimators` та `max_depth` для моделі Extra Trees. Використання Grid Search дозволило підвищити точність класифікації та автоматизувати процес налаштування моделі.

ЗАВДАННЯ 2.4

Важливість ознак

Лістинг код

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import AdaBoostRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split

housing_data = datasets.fetch_california_housing()

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.5	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

X = housing_data.data
y = housing_data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)

regressor = AdaBoostRegressor(
    DecisionTreeRegressor(max_depth=4),
    n_estimators=400,
    random_state=7
)

regressor.fit(X_train, y_train)

feature_importances = regressor.feature_importances_

feature_importances = 100.0 * (
    feature_importances / max(feature_importances)
)

feature_names = np.array(housing_data.feature_names)
index_sorted = np.flipud(np.argsort(feature_importances))
pos = np.arange(index_sorted.shape[0]) + 0.5

plt.figure(figsize=(10, 6))

plt.bar(pos, feature_importances[index_sorted], align="center")

plt.xticks(pos, feature_names[index_sorted], rotation=45)

plt.ylabel("Relative Importance")

plt.title("Feature Importance using AdaBoost")

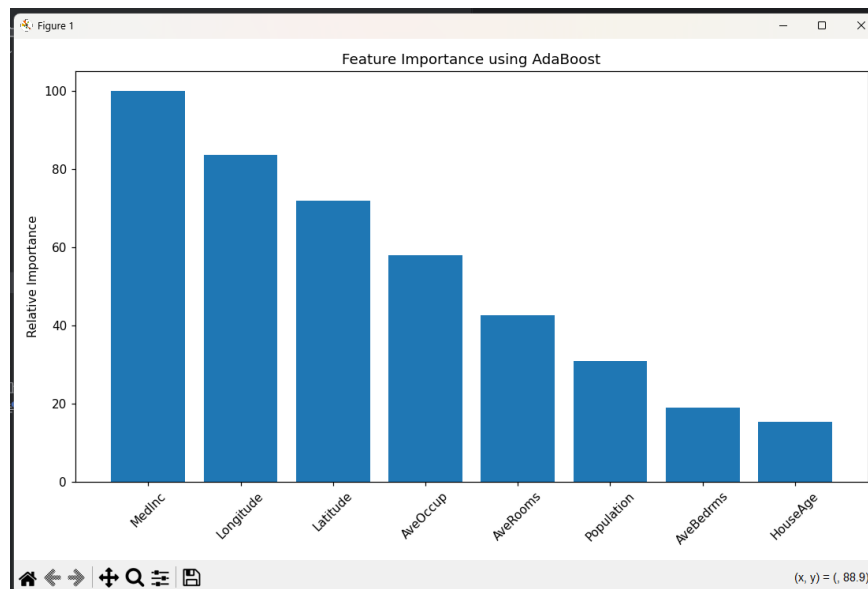
plt.tight_layout()

plt.show()

```

Результат виконання коду.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.5	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		



Висновок : У даному завданні було досліджено метод визначення відносної важливості ознак за допомогою регресора AdaBoost. Було побудовано модель для аналізу набору даних із цінами на нерухомість та визначено ознаки, які найбільше впливають на результат прогнозування. Побудована діаграма дозволила оцінити внесок кожної ознаки та визначити менш важливі параметри, якими можна знехтувати.

ЗАВДАННЯ 2.5

Прогнозування трафіку

Лістинг код

```
import numpy as np

from sklearn import preprocessing
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

input_file = "traffic_data.txt"

data = []

with open(input_file, "r") as f:
    for line in f.readlines():
        items = line[:-1].split(",")
        data.append(items)

data = np.array(data)

label_encoder = []
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.5	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

X_encoded = np.empty(data.shape)

for i, item in enumerate(data[0]):
    if item.isdigit():
        X_encoded[:, i] = data[:, i]
    else:
        label_encoder.append(preprocessing.LabelEncoder())
        X_encoded[:, i] = label_encoder[-1].fit_transform(data[:, i])

X = X_encoded[:, :-1].astype(int)
y = X_encoded[:, -1].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=5
)

regressor = ExtraTreesRegressor(
    n_estimators=100,
    max_depth=4,
    random_state=0
)

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

print("Mean absolute error:")
print(round(mean_absolute_error(y_test, y_pred), 2))

```

Результат виконання коду.

```

Mean absolute error:
7.42

```

Висновок : У ході виконання завдання було створено регресор на основі гранично випадкових лісів для прогнозування інтенсивності дорожнього руху. Було виконано попередню обробку даних, кодування нечислових ознак та навчання моделі. Отримані результати показали, що ансамблеві методи забезпечують ефективне прогнозування складних процесів на основі багатовимірних даних.

Висновок : Я використовуючи спеціалізовані бібліотеки та мову програмування Python дослідив методи ансамблів у машинному навчанні.

Посилання на [GitHub](#)

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.5	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.5	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		